

Zivilluftfahrt Datenanalyse - Mini-Projekt

Inhaltsverzeichnis

1. Team
2. Daten
3. Datenbereinigung
4. Datenvorbereitung
5. Visualisierung

1. Team

Strobl && Spitzer

2. Daten

Datensatz-Link

Quelle: <https://data.statistik.gv.at/web/catalog.jsp>

Beschreibung der Attribute

- **C-ZLFMONAT-0**: Monat im Format YYYYMM (ordinalskaliert)
- **C-ZLFE32-0**: Flughafen-Code (VIE=Wien, GRZ=Graz, INN=Innsbruck, KLU=Klagenfurt, LNZ=Linz, SZG=Salzburg) (nominal)
- **C-ZLFART-0**: Art der Bewegung (1=Linienverkehr, 2=Charterverkehr) (nominal)
- **C-ZLFR63-0**: Richtung (1=Abflug, 2=Ankunft, 3=Transit) (nominal)
- **F-ZLFFLUG**: Anzahl der Flüge (verhältnisskaliert)
- **F-ZLFPAX**: Anzahl der Passagiere (verhältnisskaliert)
- **F-ZLFFRACHT**: Fracht in kg (verhältnisskaliert)
- **F-ZLFPPOST**: Post in kg (verhältnisskaliert)
- **F-ZLFPAXEND**: Endgültige Passagiere (verhältnisskaliert)

```
In [38]: # Bibliotheken importieren
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Visualisierung konfigurieren
plt.style.use('default')
sns.set_palette("husl")
%matplotlib inline
```

```
In [39]: # Daten laden
df = pd.read_csv('./OGD_zlf_komm_ZLF_KOM_1.csv', sep=";")
print(f"Datensatz geladen mit {len(df)} Zeilen und {len(df.columns)} Spalten")
```

Datensatz geladen mit 9899 Zeilen und 9 Spalten

Initiale Standard-Analyse

```
In [40]: # Sample - Stichprobe der Daten
print("=== SAMPLE - Zufällige Stichprobe ===")
df.sample(10)
```

=== SAMPLE - Zufällige Stichprobe ===

```
Out[40]:
```

	C- ZLFMONAT-0	C- ZLFE32-0	C- ZLFART-0	C- ZLFR63-0	F- ZLFFLUG	F- ZLFPAX	F- ZLFFRACHT	ZI
2010	200411	VIE	1	3	0	1486	1499968	
5141	201208	GRZ	2	1	157	14380	0	
7563	201812	SZG	1	1	705	72734	4730	
4954	201202	KLU	2	1	26	174	0	
2734	200608	VIE	1	2	9653	716124	5593034	
2834	200610	SZG	1	2	574	45935	5048	
6104	201502	SZG	2	1	227	31346	307	
2644	200605	INN	1	2	408	15043	22870	
2029	200411	KLU	2	2	10	877	0	
1943	200409	VIE	2	3	0	2706	22115	

```
In [41]: # Head - Erste Zeilen
print("=== HEAD - Erste 10 Zeilen ===")
df.head(10)
```

=== HEAD - Erste 10 Zeilen ===

Out[41]:

	C- ZLFMONAT-0	C- ZLFE32-0	C- ZLFART-0	C- ZLFR63-0	F- ZLFFLUG	F- ZLFPAX	F- ZLFFRACHT	ZLFPC
0	200001	VIE	1	1	6351	316670	3743524	2190
1	200001	VIE	1	2	6376	325509	4000620	3650
2	200001	VIE	1	3	0	6634	150274	140
3	200001	VIE	2	1	355	20503	463078	
4	200001	VIE	2	2	346	20965	412342	
5	200001	VIE	2	3	0	2302	136417	
6	200001	GRZ	1	1	464	13887	66281	
7	200001	GRZ	1	2	447	14327	44842	
8	200001	GRZ	1	3	0	284	25	
9	200001	GRZ	2	1	50	2504	0	

```
In [42]: # Info - Datentypen und Speicher
print("=== INFO - Datenstruktur ===")
df.info()
```

```
=== INFO - Datenstruktur ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9899 entries, 0 to 9898
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   C-ZLFMONAT-0    9899 non-null  int64
1   C-ZLFE32-0      9899 non-null  object
2   C-ZLFART-0      9899 non-null  int64
3   C-ZLFR63-0      9899 non-null  int64
4   F-ZLFFLUG       9899 non-null  int64
5   F-ZLFPAX        9899 non-null  int64
6   F-ZLFFRACHT     9899 non-null  int64
7   F-ZLFPOST       9899 non-null  int64
8   F-ZLFPAXEND     9899 non-null  int64
dtypes: int64(8), object(1)
memory usage: 696.2+ KB
```

```
In [43]: # Describe - Statistische Kennzahlen
print("=== DESCRIBE - Statistische Kennzahlen ===")
df.describe()
```

```
=== DESCRIBE - Statistische Kennzahlen ===
```

Out[43]:

	C-ZLFMONAT-0	C-ZLFART-0	C-ZLFR63-0	F-ZLFFLUG	F-ZLFPAX	ZLFFRAC
count	9899.000000	9899.000000	9899.000000	9899.000000	9.899000e+03	9.899000e+
mean	201197.603091	1.518638	1.887362	729.397515	6.332161e+04	5.435357e+
std	743.305909	0.499678	0.785258	2158.038795	2.124455e+05	1.788153e+
min	200001.000000	1.000000	1.000000	0.000000	0.000000e+00	0.000000e+
25%	200512.000000	1.000000	1.000000	0.000000	1.278000e+03	0.000000e+
50%	201202.000000	2.000000	2.000000	124.000000	8.413000e+03	1.100000e+
75%	201808.000000	2.000000	3.000000	373.000000	2.248900e+04	4.100600e+
max	202511.000000	2.000000	3.000000	12274.000000	1.698293e+06	1.408562e+

In [44]:

```
# Unique Werte für kategoriale Variablen
print("=== UNIQUE WERTE ===")
print(f"\nFlughäfen (C-ZLFE32-0): {df['C-ZLFE32-0'].unique()}")
print(f"Anzahl: {df['C-ZLFE32-0'].nunique()}")

print(f"\nVerkehrsart (C-ZLFART-0): {df['C-ZLFART-0'].unique()}")
print(f"Anzahl: {df['C-ZLFART-0'].nunique()}")

print(f"\nRichtung (C-ZLFR63-0): {df['C-ZLFR63-0'].unique()}")
print(f"Anzahl: {df['C-ZLFR63-0'].nunique()}")

print(f"\nMonatswerte (C-ZLFMONAT-0): Min={df['C-ZLFMONAT-0'].min()}, Max={df['C-ZLFMONAT-0'].max()}")
print(f"Anzahl verschiedener Monate: {df['C-ZLFMONAT-0'].nunique()}")
```

```
=== UNIQUE WERTE ===
```

```
Flughäfen (C-ZLFE32-0): ['VIE' 'GRZ' 'INN' 'KLU' 'LNZ' 'SZG']
Anzahl: 6
```

```
Verkehrsart (C-ZLFART-0): [1 2]
Anzahl: 2
```

```
Richtung (C-ZLFR63-0): [1 2 3]
Anzahl: 3
```

```
Monatswerte (C-ZLFMONAT-0): Min=200001, Max=202511
Anzahl verschiedener Monate: 311
```

3. Datenbereinigung

In diesem Abschnitt werden die Daten auf Qualität geprüft und bereinigt.

In [45]:

```
# Fehlende Werte analysieren
print("=== FEHLENDE WERTE ===")
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
missing_df = pd.DataFrame({'Fehlend': missing, 'Prozent': missing_percent})
print(missing_df[missing_df['Fehlend'] > 0])

if missing.sum() == 0:
```



```
print("Keine fehlenden Werte gefunden!")
```

```
=== FEHLENDE WERTE ===  
Empty DataFrame  
Columns: [Fehlend, Prozent]  
Index: []  
Keine fehlenden Werte gefunden!
```

```
In [46]: # Falsche/ungültige Werte prüfen  
print("=== PRÜFUNG AUF UNGÜLTIGE WERTE ===")  
  
# Negative Werte bei Zählvariablen  
numeric_cols = ['F-ZLFFLUG', 'F-ZLFPAX', 'F-ZLFFRACHT', 'F-ZLFPOST', 'F-ZLFPAXEN']  
for col in numeric_cols:  
    negative_count = (df[col] < 0).sum()  
    if negative_count > 0:  
        print(f"WARNUNG: {col} hat {negative_count} negative Werte")  
    else:  
        print(f"OK: {col} hat keine negativen Werte")  
  
# Duplikate prüfen  
duplicates = df.duplicated().sum()  
print(f"\nAnzahl Duplikate: {duplicates}")
```

```
=== PRÜFUNG AUF UNGÜLTIGE WERTE ===  
OK: F-ZLFFLUG hat keine negativen Werte  
OK: F-ZLFPAX hat keine negativen Werte  
OK: F-ZLFFRACHT hat keine negativen Werte  
OK: F-ZLFPOST hat keine negativen Werte  
OK: F-ZLFPAXEND hat keine negativen Werte
```

```
Anzahl Duplikate: 0
```

```
In [47]: # Feature-Reduktion und Typsetzung  
print("=== FEATURE-REDUKTION UND TYPSETZUNG ===")  
  
# Datenkopie für Bereinigung erstellen  
df_clean = df.copy()  
  
# Kategorische Variablen als Category-Typ setzen  
df_clean['C-ZLFE32-0'] = df_clean['C-ZLFE32-0'].astype('category')  
df_clean['C-ZLFART-0'] = df_clean['C-ZLFART-0'].astype('category')  
df_clean['C-ZLFR63-0'] = df_clean['C-ZLFR63-0'].astype('category')  
  
# Monat als String behandeln für bessere Analyse  
df_clean['C-ZLFMONAT-0'] = df_clean['C-ZLFMONAT-0'].astype(str)  
  
# Jahr und Monat extrahieren für spätere Analysen  
df_clean['Jahr'] = df_clean['C-ZLFMONAT-0'].str[:4].astype(int)  
df_clean['Monat'] = df_clean['C-ZLFMONAT-0'].str[4:6].astype(int)  
  
print("Datentypen nach Bereinigung:")  
print(df_clean.dtypes)  
  
print(f"\nBereinigte Daten: {len(df_clean)} Zeilen")
```

```
=== FEATURE-REDUKTION UND TYPSETZUNG ===
```

```
Datentypen nach Bereinigung:
```

```
C-ZLFMONAT-0      object
C-ZLFE32-0        category
C-ZLFART-0        category
C-ZLFR63-0        category
F-ZLFFLUG         int64
F-ZLFPAX          int64
F-ZLFFRACHT       int64
F-ZLFPOST         int64
F-ZLFPAXEND       int64
Jahr              int64
Monat             int64
dtype: object
```

```
Bereinigte Daten: 9899 Zeilen
```

```
In [48]: # Bereinigte Daten speichern
df_clean.to_csv('korr.csv', index=False)
print("Bereinigte Daten in 'korr.csv' gespeichert")
```

```
Bereinigte Daten in 'korr.csv' gespeichert
```

4. Datenvorbereitung

Aufbereitung der Daten für verschiedene Analyseformen.

```
In [49]: # Numerische Werte extrahieren
print("=== NUMERISCHE DATEN ===")

numeric_columns = ['F-ZLFFLUG', 'F-ZLFPAX', 'F-ZLFFRACHT', 'F-ZLFPOST', 'F-ZLFPAXEND']
df_numeric = df_clean[numeric_columns].copy()

print(f"Numerischer Datensatz: {df_numeric.shape}")
print("\nStatistische Kennzahlen:")
df_numeric.describe()
```

```
=== NUMERISCHE DATEN ===
```

```
Numerischer Datensatz: (9899, 7)
```

```
Statistische Kennzahlen:
```

```
Out[49]:
```

	F-ZLFFLUG	F-ZLFPAX	F-ZLFFRACHT	F-ZLFPOST	F-ZLFPAXEND	J
count	9899.000000	9.899000e+03	9.899000e+03	9.899000e+03	9.899000e+03	9899.000
mean	729.397515	6.332161e+04	5.435357e+05	2.937304e+04	3.149128e+04	2011.911
std	2158.038795	2.124455e+05	1.788153e+06	1.196952e+05	1.536238e+05	7.433
min	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	2000.000
25%	0.000000	1.278000e+03	0.000000e+00	0.000000e+00	0.000000e+00	2005.000
50%	124.000000	8.413000e+03	1.100000e+03	0.000000e+00	0.000000e+00	2012.000
75%	373.000000	2.248900e+04	4.100600e+04	0.000000e+00	8.161500e+03	2018.000
max	12274.000000	1.698293e+06	1.408562e+07	1.176808e+06	1.673194e+06	2025.000

```
In [50]: # Numerische Daten speichern
df_numeric.to_csv('num.csv', index=False)
print("Numerische Daten in 'num.csv' gespeichert")
```

Numerische Daten in 'num.csv' gespeichert

```
In [51]: # Normalisierte Werte (Min-Max Normalisierung auf [0,1])
print("=== NORMALISIERTE DATEN ===")

denom = df_numeric.max() - df_numeric.min()
df_normalized = (df_numeric - df_numeric.min()) / denom.replace(0, 1)

print(f"Normalisierter Datensatz: {df_normalized.shape}")
print("\nStatistische Kennzahlen (normalisiert):")
df_normalized.describe()
```

=== NORMALISIERTE DATEN ===

Normalisierter Datensatz: (9899, 7)

Statistische Kennzahlen (normalisiert):

```
Out[51]:
```

	F-ZLFFLUG	F-ZLFPAX	F-ZLFFRACHT	F-ZLFPPOST	F-ZLFPAXEND	Jahr
count	9899.000000	9899.000000	9899.000000	9899.000000	9899.000000	9899.000000
mean	0.059426	0.037285	0.038588	0.024960	0.018821	0.476440
std	0.175822	0.125094	0.126949	0.101712	0.091815	0.297321
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000753	0.000000	0.000000	0.000000	0.200000
50%	0.010103	0.004954	0.000078	0.000000	0.000000	0.480000
75%	0.030389	0.013242	0.002911	0.000000	0.004878	0.720000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

```
In [52]: # Normalisierte Daten speichern
df_normalized.to_csv('norm.csv', index=False)
print("Normalisierte Daten in 'norm.csv' gespeichert")
```

Normalisierte Daten in 'norm.csv' gespeichert

```
In [53]: # Nominale Werte
print("=== NOMINALE DATEN ===")

nominal_columns = ['C-ZLFMONAT-0', 'C-ZLFE32-0', 'C-ZLFART-0', 'C-ZLFR63-0']
df_nominal = df_clean[nominal_columns].copy()

print(f"Nominaler Datensatz: {df_nominal.shape}")
print("\nWertehäufigkeiten:")
for col in nominal_columns:
    print(f"\n{col}:")
    print(df_nominal[col].value_counts().head())
```

```
=== NOMINALE DATEN ===
```

```
Nominaler Datensatz: (9899, 4)
```

```
Wertehäufigkeiten:
```

```
C-ZLFMONAT-0:
```

```
C-ZLFMONAT-0
```

```
200208      36
```

```
200105      36
```

```
200310      36
```

```
200404      36
```

```
200508      36
```

```
Name: count, dtype: int64
```

```
C-ZLFE32-0:
```

```
C-ZLFE32-0
```

```
VIE      1864
```

```
LNZ      1742
```

```
GRZ      1697
```

```
SZG      1657
```

```
INN      1584
```

```
Name: count, dtype: int64
```

```
C-ZLFART-0:
```

```
C-ZLFART-0
```

```
2      5134
```

```
1      4765
```

```
Name: count, dtype: int64
```

```
C-ZLFR63-0:
```

```
C-ZLFR63-0
```

```
1      3672
```

```
2      3670
```

```
3      2557
```

```
Name: count, dtype: int64
```

```
In [54]: # Nominale Daten speichern
df_nominal.to_csv('nom.csv', index=False)
print("Nominale Daten in 'nom.csv' gespeichert")
```

```
Nominale Daten in 'nom.csv' gespeichert
```

5. Visualisierung mit Seaborn

Umfassende Visualisierung der Daten mit verschiedenen Diagrammtypen.

```
In [55]: # Eigene Farbpalette definieren
custom_colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A', '#98D8C8', '#F7DC6F']
sns.set_palette(custom_colors)

print("Eigene Farbpalette definiert")
sns.palplot(sns.color_palette(custom_colors))
plt.title('Eigene Farbpalette')
plt.show()
```

```
Eigene Farbpalette definiert
```

Eigene Farbpalette



```
In [56]: # Korrelationsmatrix mit matplotlib.pyplot.matshow()
print("=== KORRELATION - Matplotlib matshow() ===")

correlation = df_numeric.corr()

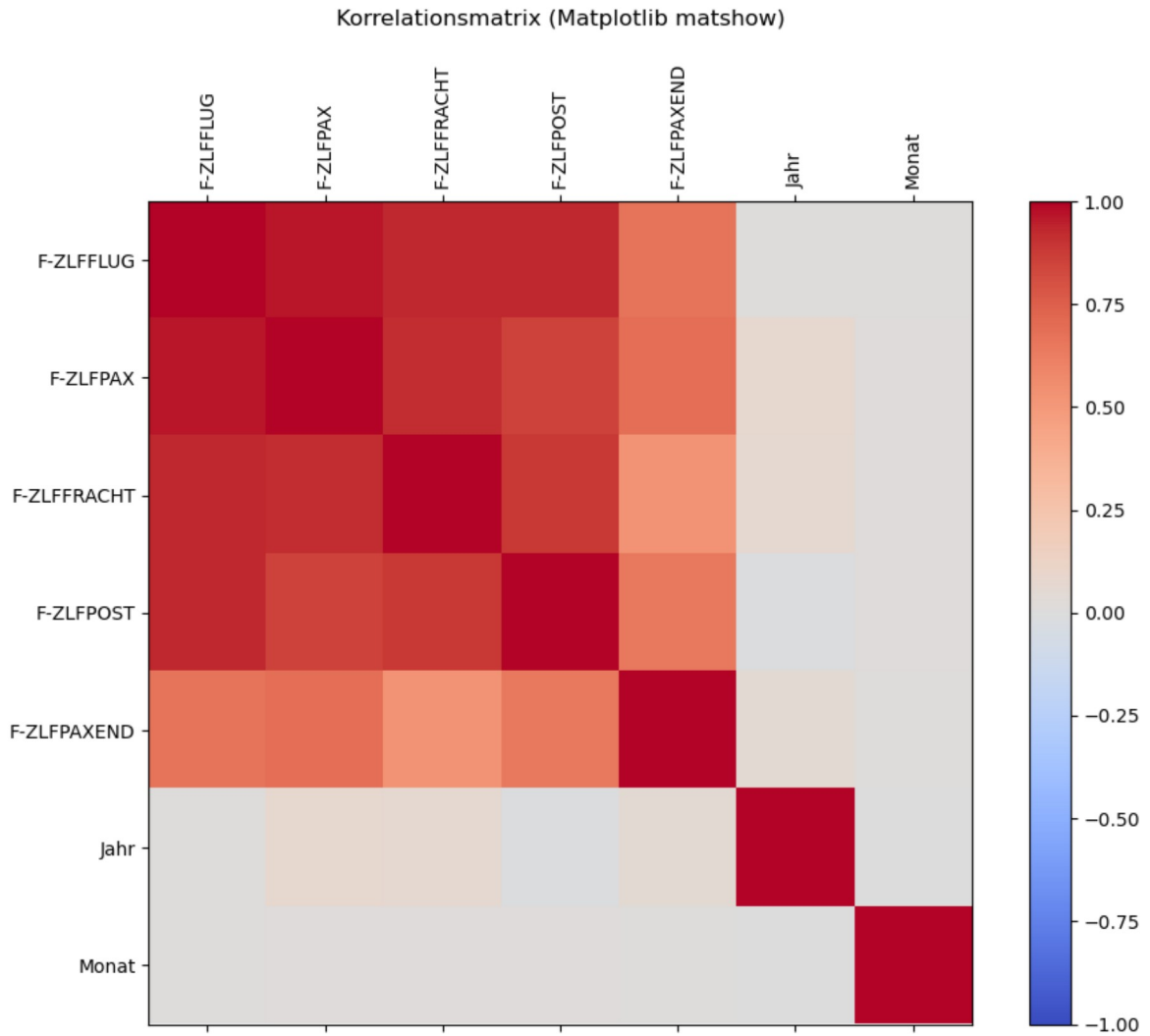
fig, ax = plt.subplots(figsize=(10, 8))
cax = ax.matshow(correlation, cmap='coolwarm', vmin=-1, vmax=1)
fig.colorbar(cax)

# Achsenbeschriftungen
ax.set_xticks(range(len(correlation.columns)))
ax.set_yticks(range(len(correlation.columns)))
ax.set_xticklabels(correlation.columns, rotation=90)
ax.set_yticklabels(correlation.columns)

plt.title('Korrelationsmatrix (Matplotlib matshow)', pad=20)
plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Starke Korrelation zwischen F-ZLFPAX und F-ZLFPAXEND (erwartbar, da eng
print("- Moderate Korrelation zwischen Flügen und Passagieren")
print("- Fracht und Post zeigen schwächere Korrelationen zu Passagierzahlen")

=== KORRELATION - Matplotlib matshow() ===
```

Auffälligkeiten:

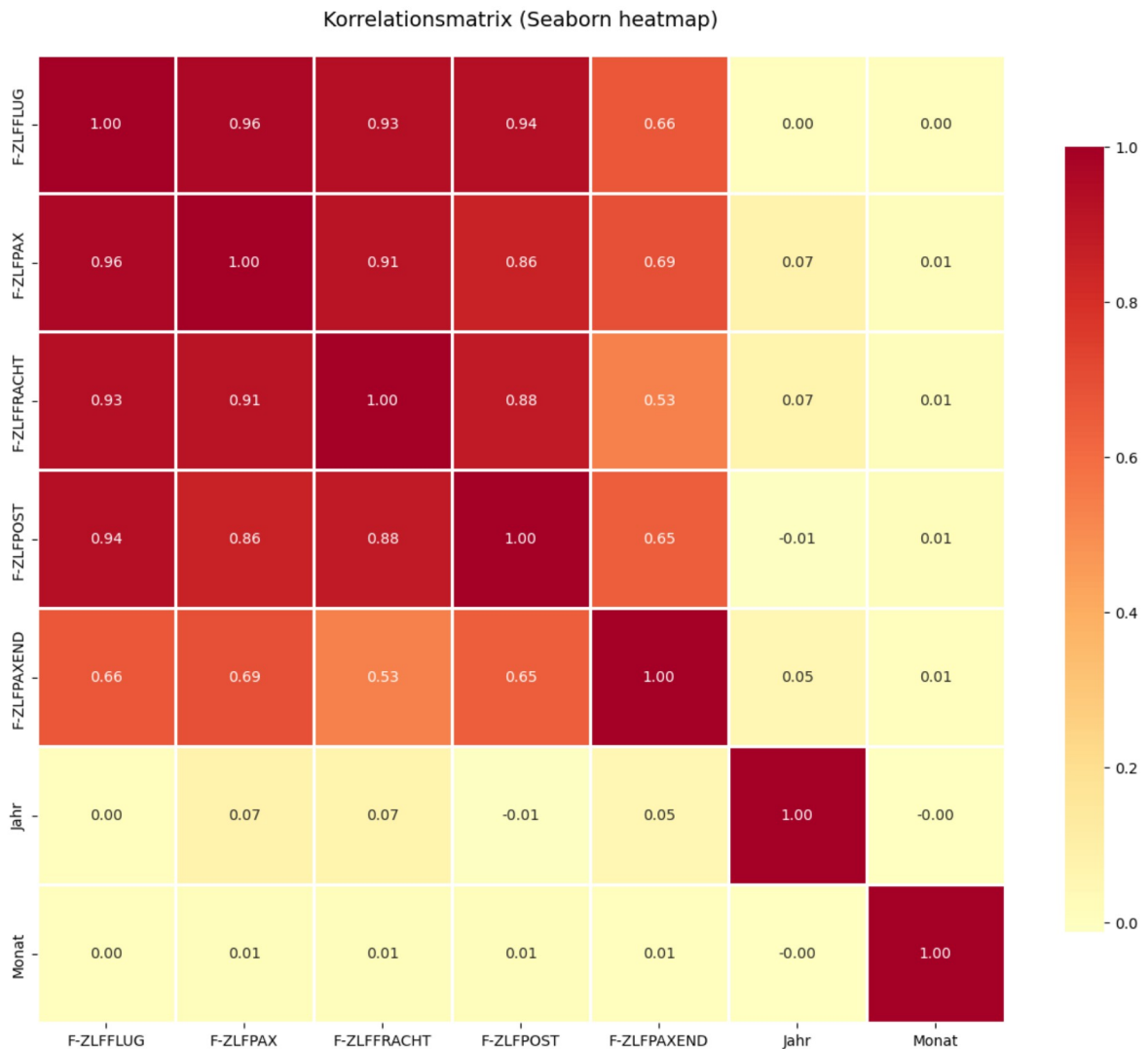
- Starke Korrelation zwischen F-ZLFPAX und F-ZLFPAXEND (erwartbar, da eng verwandt)
- Moderate Korrelation zwischen Flügen und Passagieren
- Fracht und Post zeigen schwächere Korrelationen zu Passagierzahlen

```
In [57]: # Korrelationsmatrix mit seaborn.heatmap()
print("=== KORRELATION - Seaborn heatmap() ===")

plt.figure(figsize=(12, 10))
sns.heatmap(correlation, annot=True, fmt='.2f', cmap='RdYlBu_r',
            center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Korrelationsmatrix (Seaborn heatmap)', fontsize=14, pad=20)
plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Die Heatmap zeigt die Korrelationswerte deutlicher mit Farbcodierung")
print("- Negative Korrelationen sind in Blautönen, positive in Rottönen dargestellt")
print("- Jahr zeigt teilweise negative Korrelationen, was auf Trends hindeutet")

=== KORRELATION - Seaborn heatmap() ===
```

Auffälligkeiten:

- Die Heatmap zeigt die Korrelationswerte deutlicher mit Farbcodierung
- Negative Korrelationen sind in Blautönen, positive in Rottönen dargestellt
- Jahr zeigt teilweise negative Korrelationen, was auf Trends hindeutet

```
In [58]: # Pairplot - Für bessere Performance nur Subset
print("=== PAIRPLOT ===")

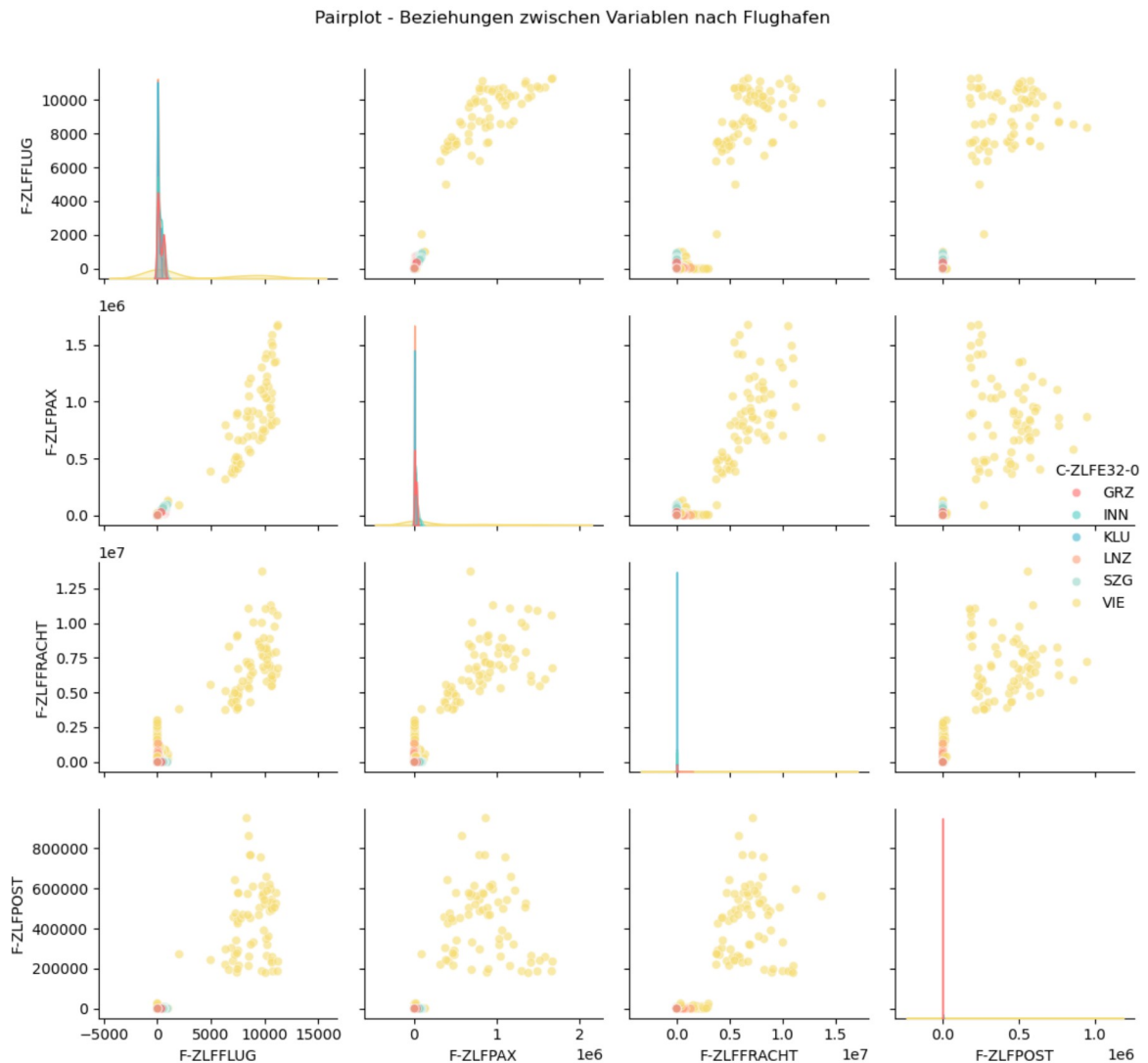
# Auswahl wichtiger numerischer Variablen
pairplot_cols = ['F-ZLFFLUG', 'F-ZLFPAX', 'F-ZLFFRACHT', 'F-ZLFPOST']
df_pairplot = df_clean[pairplot_cols + ['C-ZLFE32-0']].copy()

# Sample für Performance (jeder 10. Eintrag)
df_pairplot_sample = df_pairplot.iloc[::10]

sns.pairplot(df_pairplot_sample, hue='C-ZLFE32-0', palette=custom_colors,
              diag_kind='kde', plot_kws={'alpha': 0.6})
plt.suptitle('Pairplot - Beziehungen zwischen Variablen nach Flughafen', y=1.01)
plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- VIE (Wien) zeigt deutlich höhere Werte in allen Kategorien")
print("- Starke lineare Beziehung zwischen Flügen und Passagieren sichtbar")
print("- Fracht und Post zeigen weniger konsistente Muster")

=== PAIRPLOT ===
```



Auffälligkeiten:

- VIE (Wien) zeigt deutlich höhere Werte in allen Kategorien
- Starke lineare Beziehung zwischen Flügen und Passagieren sichtbar
- Fracht und Post zeigen weniger konsistente Muster

```
In [59]: # Box- vs Violinplot mit Subplots
print("=== BOX- VS VIOLINPLOT ===")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Boxplot - Passagiere nach Flughafen
sns.boxplot(data=df_clean, x='C-ZLFE32-0', y='F-ZLFPAX', ax=axes[0, 0], palette=
axes[0, 0].set_title('Boxplot - Passagiere nach Flughafen', fontsize=12)
axes[0, 0].set_xlabel('Flughafen')
axes[0, 0].set_ylabel('Passagiere')

# Violinplot - Passagiere nach Flughafen
sns.violinplot(data=df_clean, x='C-ZLFE32-0', y='F-ZLFPAX', ax=axes[0, 1], palet
axes[0, 1].set_title('Violinplot - Passagiere nach Flughafen', fontsize=12)
axes[0, 1].set_xlabel('Flughafen')
axes[0, 1].set_ylabel('Passagiere')

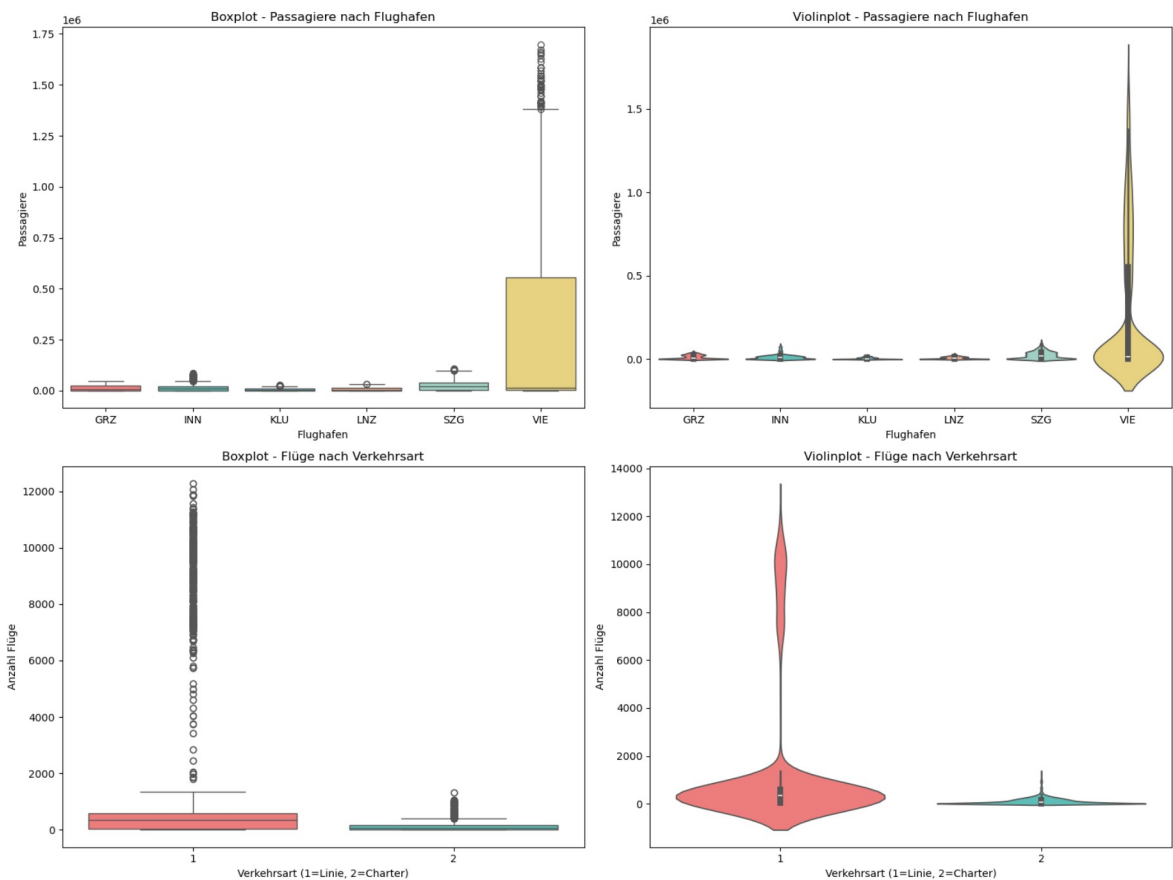
# Boxplot - Flüge nach Verkehrsart
sns.boxplot(data=df_clean, x='C-ZLFART-0', y='F-ZLFFLUG', ax=axes[1, 0], palette
axes[1, 0].set_title('Boxplot - Flüge nach Verkehrsart', fontsize=12)
axes[1, 0].set_xlabel('Verkehrsart (1=Linie, 2=Charter)')
axes[1, 0].set_ylabel('Anzahl Flüge')
```

```
# Violinplot - Flüge nach Verkehrsart
sns.violinplot(data=df_clean, x='C-ZLFART-0', y='F-ZLFFLUG', ax=axes[1, 1], pale
axes[1, 1].set_title('Violinplot - Flüge nach Verkehrsart', fontsize=12)
axes[1, 1].set_xlabel('Verkehrsart (1=Linie, 2=Charter)')
axes[1, 1].set_ylabel('Anzahl Flüge')

plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Boxplots zeigen Median, Quartile und Ausreißer deutlich")
print("- Violinplots zeigen die Verteilungsdichte besser")
print("- VIE hat deutlich höhere Passagierzahlen und größere Variabilität")
print("- Linienverkehr zeigt mehr Flüge als Charterverkehr")
print("- Viele Ausreißer bei kleineren Flughäfen sichtbar")
```

=== BOX- VS VIOLINPLOT ===



Auffälligkeiten:

- Boxplots zeigen Median, Quartile und Ausreißer deutlich
- Violinplots zeigen die Verteilungsdichte besser
- VIE hat deutlich höhere Passagierzahlen und größere Variabilität
- Linienverkehr zeigt mehr Flüge als Charterverkehr
- Viele Ausreißer bei kleineren Flughäfen sichtbar

```
In [60]: # Histogramme und KDE (Kernel Density Estimation)
print("=== HISTOGRAMME UND KDE ===")

fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# Histogramm - Passagiere
axes[0, 0].hist(df_clean['F-ZLFPAX'], bins=50, color=custom_colors[0], alpha=0.7)
axes[0, 0].set_title('Histogramm - Passagiere', fontsize=12)
axes[0, 0].set_xlabel('Passagiere')
axes[0, 0].set_ylabel('Häufigkeit')
```

```

axes[0, 0].grid(alpha=0.3)

# KDE - Passagiere
df_clean['F-ZLFPAX'].plot.kde(ax=axes[0, 1], color=custom_colors[1], linewidth=2)
axes[0, 1].set_title('KDE - Passagiere', fontsize=12)
axes[0, 1].set_xlabel('Passagiere')
axes[0, 1].set_ylabel('Dichte')
axes[0, 1].grid(alpha=0.3)

# Histogramm mit KDE - Fracht
sns.histplot(data=df_clean, x='F-ZLFFRACHT', kde=True, color=custom_colors[2], a
axes[1, 0].set_title('Histogramm mit KDE - Fracht', fontsize=12)
axes[1, 0].set_xlabel('Fracht (kg)')
axes[1, 0].set_ylabel('Häufigkeit')

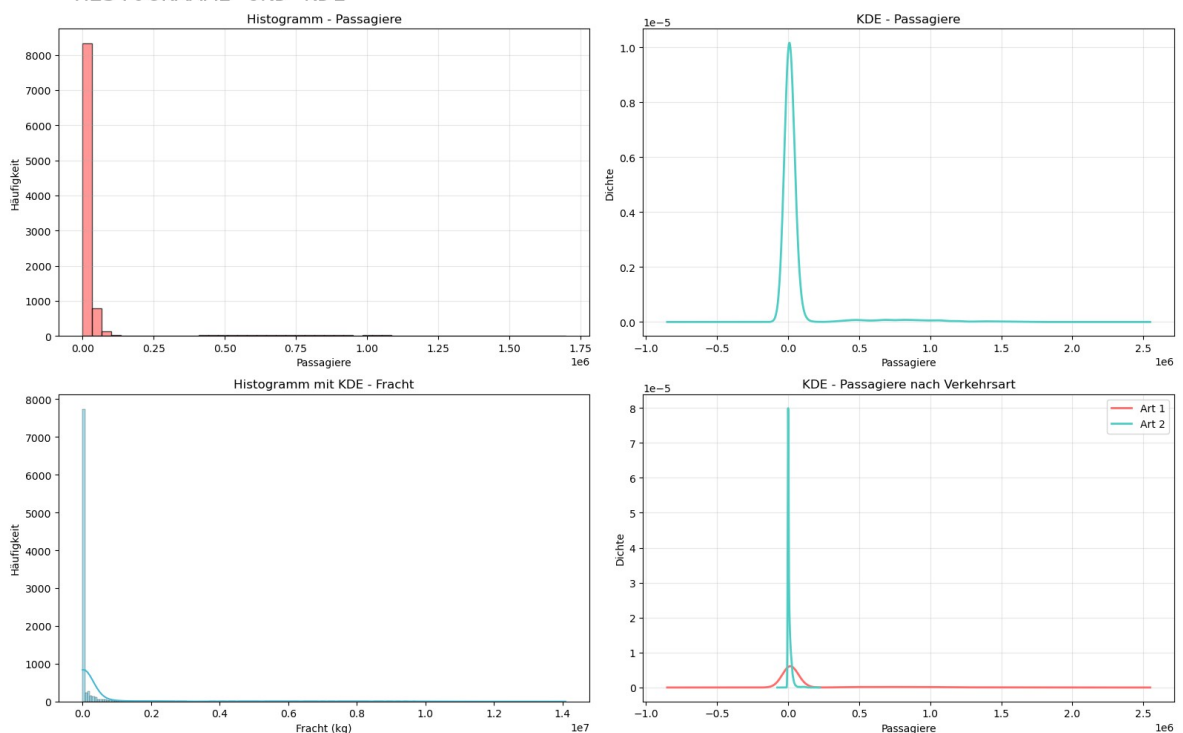
# Mehrere KDE-Plots - Passagiere nach Verkehrsart
for art in df_clean['C-ZLFART-0'].unique():
    subset = df_clean[df_clean['C-ZLFART-0'] == art]
    subset['F-ZLFPAX'].plot.kde(ax=axes[1, 1], label=f"Art {art}", linewidth=2)
axes[1, 1].set_title('KDE - Passagiere nach Verkehrsart', fontsize=12)
axes[1, 1].set_xlabel('Passagiere')
axes[1, 1].set_ylabel('Dichte')
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Passagierzahlen zeigen eine rechtsschiefe Verteilung")
print("- Viele Flüge mit niedrigen Passagierzahlen, wenige mit sehr hohen")
print("- Fracht zeigt ähnliche Verteilung mit Konzentration bei niedrigen Werten")
print("- Linien- und Charterverkehr haben unterschiedliche Verteilungsprofile")

```

=== HISTOGRAMME UND KDE ===



Auffälligkeiten:

- Passagierzahlen zeigen eine rechtsschiefe Verteilung
- Viele Flüge mit niedrigen Passagierzahlen, wenige mit sehr hohen
- Fracht zeigt ähnliche Verteilung mit Konzentration bei niedrigen Werten
- Linien- und Charterverkehr haben unterschiedliche Verteilungsprofile

```
In [61]: # Scatterplots
print("=== SCATTERPLOTS ===")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

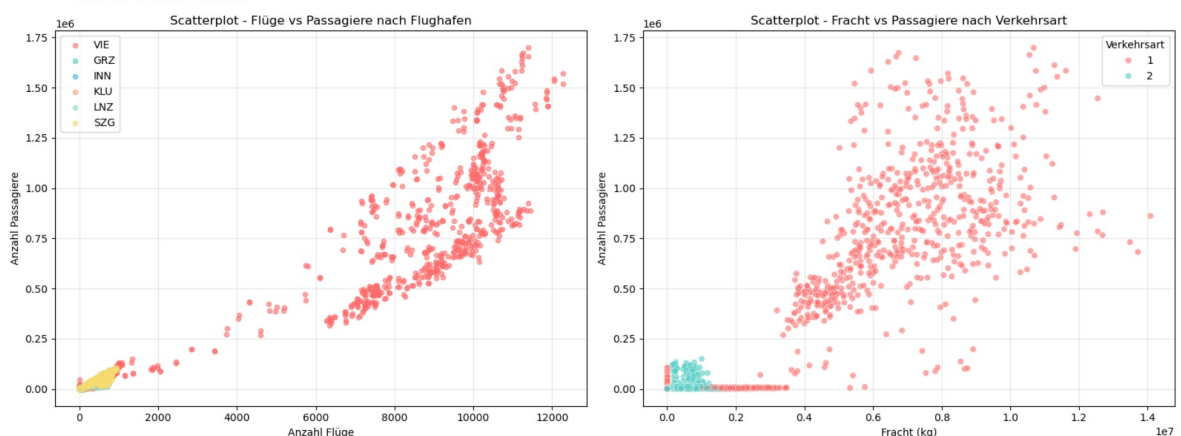
# Scatter - Flüge vs Passagiere nach Flughafen
for i, airport in enumerate(df_clean['C-ZLFE32-0'].unique()):
    subset = df_clean[df_clean['C-ZLFE32-0'] == airport]
    axes[0].scatter(subset['F-ZLFFLUG'], subset['F-ZLFPAX'],
                    label=airport, alpha=0.6, s=20, color=custom_colors[i % len(c
axes[0].set_title('Scatterplot - Flüge vs Passagiere nach Flughafen', fontsize=1
axes[0].set_xlabel('Anzahl Flüge')
axes[0].set_ylabel('Anzahl Passagiere')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Scatter - Fracht vs Passagiere
sns.scatterplot(data=df_clean, x='F-ZLFFRACHT', y='F-ZLFPAX',
                hue='C-ZLFART-0', palette=custom_colors, alpha=0.6, ax=axes[1])
axes[1].set_title('Scatterplot - Fracht vs Passagiere nach Verkehrsart', fontsiz
axes[1].set_xlabel('Fracht (kg)')
axes[1].set_ylabel('Anzahl Passagiere')
axes[1].legend(title='Verkehrsart')
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Klarer linearer Zusammenhang zwischen Flügen und Passagieren")
print("- VIE dominiert mit höchsten Werten in beiden Dimensionen")
print("- Fracht und Passagiere zeigen schwächeren Zusammenhang")
print("- Einige Flüge mit hoher Fracht aber wenigen Passagieren (Cargo-Flüge)")
```

=== SCATTERPLOTS ===



Auffälligkeiten:

- Klarer linearer Zusammenhang zwischen Flügen und Passagieren
- VIE dominiert mit höchsten Werten in beiden Dimensionen
- Fracht und Passagiere zeigen schwächeren Zusammenhang
- Einige Flüge mit hoher Fracht aber wenigen Passagieren (Cargo-Flüge)


```
In [62]: # Linien- und Balkendiagramme
print("=== LINIEN- UND BALKENDIAGRAMME ===")

# Daten aggregieren
passengers_by_airport = df_clean.groupby('C-ZLFE32-0')['F-ZLFPAX'].sum().sort_val
passengers_by_year = df_clean.groupby('Jahr')['F-ZLFPAX'].sum()

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Balkendiagramm - Passagiere nach Flughafen
passengers_by_airport.plot(kind='bar', ax=axes[0, 0], color=custom_colors)
axes[0, 0].set_title('Balkendiagramm - Gesamtpassagiere nach Flughafen', fontsize=12)
axes[0, 0].set_xlabel('Flughafen')
axes[0, 0].set_ylabel('Gesamtanzahl Passagiere')
axes[0, 0].tick_params(axis='x', rotation=0)
axes[0, 0].grid(axis='y', alpha=0.3)

# Liniendiagramm - Passagiere über Zeit
passengers_by_year.plot(kind='line', ax=axes[0, 1], color=custom_colors[0], line_color=custom_colors[0])
axes[0, 1].set_title('Liniendiagramm - Passagiere über die Jahre', fontsize=12)
axes[0, 1].set_xlabel('Jahr')
axes[0, 1].set_ylabel('Gesamtanzahl Passagiere')
axes[0, 1].grid(alpha=0.3)

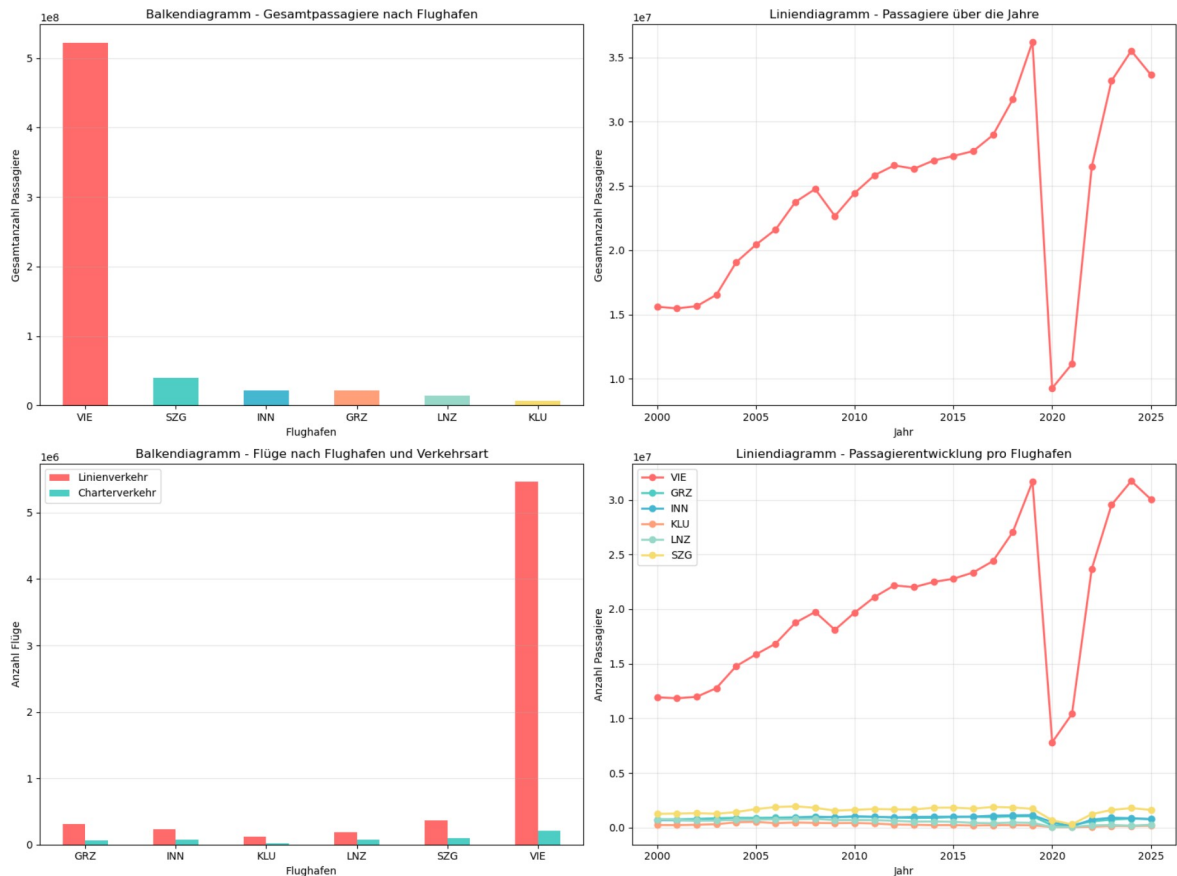
# Gruppiertes Balkendiagramm - Flüge nach Flughafen und Verkehrsart
flights_grouped = df_clean.groupby(['C-ZLFE32-0', 'C-ZLFART-0'])['F-ZLFFLUG'].sum()
flights_grouped.plot(kind='bar', ax=axes[1, 0], color=custom_colors[:2])
axes[1, 0].set_title('Balkendiagramm - Flüge nach Flughafen und Verkehrsart', fontsize=12)
axes[1, 0].set_xlabel('Flughafen')
axes[1, 0].set_ylabel('Anzahl Flüge')
axes[1, 0].legend(['Linienverkehr', 'Charterverkehr'])
axes[1, 0].tick_params(axis='x', rotation=0)
axes[1, 0].grid(axis='y', alpha=0.3)

# Liniendiagramm - Entwicklung pro Flughafen
for i, airport in enumerate(df_clean['C-ZLFE32-0'].unique()):
    subset = df_clean[df_clean['C-ZLFE32-0'] == airport]
    pax_by_year = subset.groupby('Jahr')['F-ZLFPAX'].sum()
    pax_by_year.plot(kind='line', ax=axes[1, 1], label=airport,
                      linewidth=2, marker='o', color=custom_colors[i % len(custom_colors)])
axes[1, 1].set_title('Liniendiagramm - Passagierentwicklung pro Flughafen', fontsize=12)
axes[1, 1].set_xlabel('Jahr')
axes[1, 1].set_ylabel('Anzahl Passagiere')
axes[1, 1].legend()
axes[1, 1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- VIE dominiert mit Abstand bei Passagierzahlen")
print("- Deutlicher Einbruch in den Jahren 2020-2021 (COVID-19 Pandemie)")
print("- Linienverkehr macht den größten Anteil der Flüge aus")
print("- Alle Flughäfen zeigen ähnliche Trendmuster über die Zeit")
print("- Sichtbare Erholung ab 2022")
```

=== LINIEN- UND BALKENDIAGRAMME ===



Auffälligkeiten:

- VIE dominiert mit Abstand bei Passagierzahlen
- Deutlicher Einbruch in den Jahren 2020-2021 (COVID-19 Pandemie)
- Linienverkehr macht den größten Anteil der Flüge aus
- Alle Flughäfen zeigen ähnliche Trendmuster über die Zeit
- Sichtbare Erholung ab 2022

```
In [63]: # Zusätzliche Visualisierungen
print("=== ZUSÄTZLICHE ANALYSEN ===")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

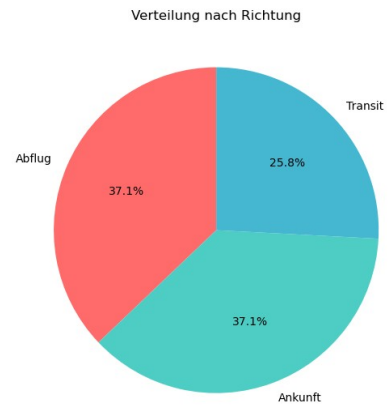
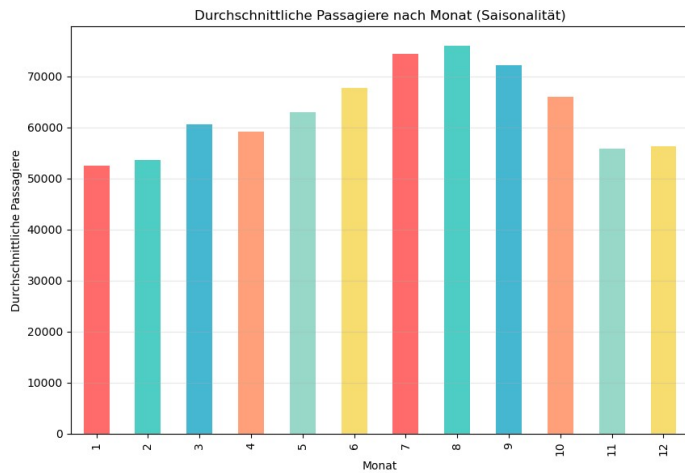
# Monatliche Saisonalität
monthly_pax = df_clean.groupby('Monat')['F-ZLFPAX'].mean()
monthly_pax.plot(kind='bar', ax=axes[0], color=custom_colors)
axes[0].set_title('Durchschnittliche Passagiere nach Monat (Saisonalität)', font
axes[0].set_xlabel('Monat')
axes[0].set_ylabel('Durchschnittliche Passagiere')
axes[0].grid(axis='y', alpha=0.3)

# Richtungsverteilung
direction_counts = df_clean['C-ZLFR63-0'].value_counts()
axes[1].pie(direction_counts, labels=['Abflug', 'Ankunft', 'Transit'],
            autopct='%1.1f%%', colors=custom_colors[:3], startangle=90)
axes[1].set_title('Verteilung nach Richtung', fontsize=12)

plt.tight_layout()
plt.show()

print("\nAuffälligkeiten:")
print("- Saisonale Schwankungen erkennbar mit Spitzen in Sommermonaten")
print("- Abflug und Ankunft sind etwa gleich verteilt (wie erwartet)")
print("- Transit macht einen kleinen Teil aus")
```

=== ZUSÄTZLICHE ANALYSEN ===



Auffälligkeiten:

- Saisonale Schwankungen erkennbar mit Spitzen in Sommermonaten
- Abflug und Ankunft sind etwa gleich verteilt (wie erwartet)
- Transit macht einen kleinen Teil aus

Zusammenfassung

Haupterkenntnisse:

1. **Datenqualität:** Der Datensatz ist vollständig ohne fehlende Werte und ohne fehlerhafte Einträge

2. **Flughäfen:**

- VIE (Wien) ist der mit Abstand größte Flughafen
- Alle anderen Flughäfen haben deutlich geringeres Verkehrsaufkommen

3. **Korrelationen:**

- Starke Korrelation zwischen Flügen und Passagieren
- Fracht und Post zeigen unabhängigere Muster

4. **Zeitliche Trends:**

- Deutlicher COVID-19 Einbruch 2020-2021
- Saisonale Schwankungen mit Sommerspitzen
- Erholung ab 2022 sichtbar

5. **Verkehrsarten:**

- Linienverkehr dominiert über Charterverkehr
- Unterschiedliche Verteilungsmuster bei Passagierzahlen

Gespeicherte Dateien:

- `korr.csv` : Bereinigte Daten
- `num.csv` : Numerische Werte
- `norm.csv` : Normalisierte Werte
- `nom.csv` : Nominale Werte